

Nama : Radhit Akriandra
NRP : 5025251084
Praktikum 4

Soal 1: Trailblazer

Penjelasan

Pak Arsud memiliki n peliharaan alien yang teridentifikasi **blob**. Para blob akan melakukan patroli di dalam area dengan r baris dan c kolom. Masing-masing blob akan diberi *cmd* atau yang merepresentasikan jalur yang dilalui setiap blob.

Setiap *cmd* memiliki karakter 'W', 'A', 'S', 'D' yang merepresentasikan arah gerak blob yaitu,

- W: Blob bergerak dari (i, j) ke $(i - 1, j)$.
- A: Blob bergerak dari (i, j) ke $(i, j - 1)$.
- S: Blob bergerak dari (i, j) ke $(i + 1, j)$.
- D: Blob bergerak dari (i, j) ke $(i, j + 1)$.

Setiap blob memiliki nilai x jejak. Jika sebuah blob berada di koordinat (i, j) , maka koordinat tersebut bertambah sebanyak x jejak. Setiap blob berpindah maka nilai x sesuai aturan berikut.

$$x = \begin{cases} 3x + 1, & \text{jika } x \text{ ganjil,} \\ \frac{x}{2}, & \text{jika } x \text{ genap.} \end{cases}$$

Pak Arsud akan memberikan q aba-aba pada blob pada indeks q_i untuk memulai perintahnya. Keluarkan kondisi area setelah q aba-aba diberikan.

Solusi

Untuk setiap koordinat blob, kita akan tambahkan x jejak awal. Selanjutnya kita akan lakukan pergerakan *cmd* untuk setiap aba-aba pada blob ke- q_i . Untuk setiap pergerakan, kita akan perbarui nilai x menggunakan fungsi **change** dan tambahkan x terbaru ke area pada koordinat tersebut. Setelah selesai melakukan pergerakan *cmd*, *update* nilai r dan c untuk *checkpoint* jika seandainya blob tersebut akan bergerak lagi.

Berikut kode untuk soal di atas.

trailblazer.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int r, c, x, cmdlen;
    char cmd[101];
} Blob;

void change(int *x){
    if((*x) & 1){
        *x = 3*(*x) + 1;
    }else{
        *x /= 2;
    }
}
```

```

    }
}

int main(){
    int n, r, c;
    scanf("%d %d %d", &n, &r, &c);
    int **grid = (int**)malloc(r * sizeof(int*));
    for(int i = 0; i < r; i++){
        grid[i] = (int*)calloc(c, sizeof(int));
    }
    Blob *blob = (Blob*)malloc(n * sizeof(Blob));
    for(int i = 0; i < n; i++){
        scanf("%d %d %d %d %s", &blob[i].r, &blob[i].c, &blob[i].x, &blob[i].cmdlen,
            blob[i].cmd);
        int x = blob[i].r;
        int y = blob[i].c;
        int val = blob[i].x;
        grid[x][y] += val;
    }
    int query;
    scanf("%d", &query);
    for(int i = 0; i < query; i++){
        int q;
        scanf("%d", &q);
        int len = blob[q].cmdlen;
        int nowx = blob[q].r, nowy = blob[q].c;
        for(int j = 0; j < len; j++){
            char cmd = blob[q].cmd[j];
            if(cmd == 'W'){
                nowx--;
            }else if(cmd == 'S'){
                nowx++;
            }else if(cmd == 'A'){
                nowy--;
            }else if(cmd == 'D'){
                nowy++;
            }
            change(&blob[q].x);
            if(nowx >= 0 && nowx < r && nowy >= 0 && nowy < c){
                grid[nowx][nowy] += blob[q].x;
            }
        }
        blob[q].r = nowx;
        blob[q].c = nowy;
    }
    for(int i = 0; i < r; i++){
        for(int j = 0; j < c; j++){
            printf("%d ", grid[i][j]);
        }
        printf("\n");
    }
    for(int i = 0; i < r; i++){
        free(grid[i]);
    }
    free(blob);
    free(grid);
    return 0;
}

```

Soal 2: Malas

Penjelasan

Lakukan tugas sesuai dengan fungsi yaitu,

1. "ADD" untuk menambahkan tugas baru beserta nilai prioritasnya.
2. "CHECK" untuk menampilkan detail tugas pada indeks input.
3. "LIST" untuk menampilkan semua tugas yang ada.
4. "CLEAR" untuk menyelesaikan dan menghapus tugas pada indeks input.

Solusi

Soal ini cukup melakukan alokasi task secara dinamis karena jumlah task yang dapat berubah tergantung user input. Sehingga cukup untuk mengimplementasikan logika pada soal.

Berikut kode untuk soal di atas.

malas.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct {
    char name[51];
    int priority;
} task;

int main(){
    task *taskPtr = NULL;
    int index = 0;
    while(1){
        char cmd[20];
        scanf(" %s", cmd);
        if(!strcmp(cmd, "EXIT")){
            break;
        }else if(!strcmp(cmd, "ADD")){
            taskPtr = (task*)realloc(taskPtr, (index + 1) * sizeof(task));
            scanf("%s %d", taskPtr[index].name, &taskPtr[index].priority);
            printf("%s %d is added\n", taskPtr[index].name, taskPtr[index].priority);
            index++;
        }else if(!strcmp(cmd, "CHECK")){ // cek index tersebut ada tugas atau sudah dihapus
            int id;
            scanf("%d", &id);
            if(id < index && strcmp(taskPtr[id].name, "[deleted]")){
                printf("%s %d\n", taskPtr[id].name, taskPtr[id].priority);
            }else{
                printf("Task at ID %d is empty\n", id);
            }
        }else if(!strcmp(cmd, "LIST")){
            printf("TO DO LIST:\n");
            for(int i = 0; i < index; i++){
                if(strcmp(taskPtr[i].name, "[deleted]")){ // skip deleted
                    printf("%s %d\n", taskPtr[i].name, taskPtr[i].priority);
                }
            }
        }else if(!strcmp(cmd, "CLEAR")){
            int id;
```

```

        scanf("%d", &id);
        printf("Task %s is cleared\n", taskPtr[id].name);
        strcpy(taskPtr[id].name, "[deleted]");
    }
}
free(taskPtr);
return 0;
}

```

Soal 3: Aku Peramal

Penjelasan

Diberikan angka ramalan dua bulan pertama yaitu A_1 dan A_2 . Berapakah angka ramalan pada bulan ke- x jika pola ramalan mengikuti pola matematika tertentu.

Solusi

Pola dari soal adalah barisan Fibonacci, di mana suku berikutnya adalah penjumlahan dua suku sebelumnya. Karena soal meminta kita menggunakan template yang ada, kita akan menghitung barisan Fibonacci secara iteratif yaitu dengan menyimpan suatu variabel `next` yang berisi penjumlahan dua suku sebelumnya. Kemudian kita akan lakukan *swap* terhadap suku pertama dengan suku selanjutnya.

Berikut kode untuk soal di atas.

peramal.c

```

#include <stdio.h>

void magician(long long *a1, long long *a2, long long *x, long long *ans){
    if(*x == 1){
        *ans = *a1;
        return;
    }
    if(*x == 2){
        *ans = *a2;
        return;
    }
    long long next;
    for(int i = 3; i <= *x; i++){
        next = *a1 + *a2;
        *a1 = *a2;
        *a2 = next;
    }
    *ans = next;
}

int main() {
    long long t;
    scanf("%lld", &t);
    while(t--){
        long long a1, a2, x;
        scanf("%lld %lld %lld", &a1, &a2, &x);
        long long ans;
        magician(&a1, &a2, &x, &ans);
        printf("%lld\n", ans);
    }
}

```

```
    return 0;
}
```

Soal 4: Banyak Banget Connection

Penjelasan

Kasane Teto terjebak pada rangkaian pulau-pulau yang terindeks dari 1 sampai N . Setiap pulau terkoneksi satu sama lain melalui jembatan. Keluarkan setiap pulau ke- i , lalu keluarkan pulau-pulau yang terkoneksi dengan pulau ke- i .

Solusi

Soal ini meminta kita untuk mencetak representasi suatu graf dengan N node menggunakan *adjacency list* di mana *adjacency list* adalah suatu representasi graf (objek yang terkoneksi) yang menyimpan informasi tetangga dari setiap *node*. Representasi *adjacency list* membutuhkan array dinamis dikarenakan jumlah tetangga tiap node tidak sama. Sehingga kita perlu bikin sebuah array dinamis 2D menggunakan *struct* yang menyimpan ukuran yang dibutuhkan, ukuran maksimum, dan array yang berisi tetangga setiap *node*.

Fungsi `add_edge` akan menambahkan tetangga setiap *node* dan mengalokasikan ukuran maksimum sebanyak 2 kali dari ukuran yang dibutuhkan. Karena koneksi antarpohon bersifat *undirected*, maka kita akan lakukan penambahan koneksi secara dua arah.

Berikut kode untuk soal di atas.

adjlist.c

```
#include <stdio.h>
#include <stdlib.h>

const int mxn = 2e5;
typedef struct {
    int curr_size;
    int max_size;
    int *data;
} Vector;

Vector adj[200005];

void add_edge(Vector *adj, int to){
    if(adj->curr_size == adj->max_size){
        if(adj->curr_size == 0){
            adj->max_size = 1;
        }else{
            adj->max_size = 2*(adj->curr_size);
        }
        adj->data = (int *)realloc(adj->data, (adj->max_size) * sizeof(int));
    }
    adj->data[adj->curr_size] = to;
    adj->curr_size++;
}

int main(){
    int u, v, n = 1;
    for(int i = 0; i < mxn; i++){
```

```

    adj[i].curr_size = 0;
    adj[i].max_size = 0;
    adj[i].data = NULL;
}
while(scanf("%d %d", &u, &v) != EOF){
    add_edge(&adj[u], v);
    add_edge(&adj[v], u);
    if(n < u){
        n = u;
    }
    if(n < v){
        n = v;
    }
}
for(int i = 1; i <= n; i++){
    if(adj[i].curr_size == 0) continue;
    printf("Pulau %d: ", i);
    for(int j = 0; j < adj[i].curr_size; j++){
        printf("%d ", adj[i].data[j]);
    }
    printf("\n");
}
for(int i = 1; i <= n; i++){
    if(adj[i].data != NULL){
        free(adj[i].data);
    }
}
}

```